

SCSI Pass-Through (SPT) Update (TOI)

by

Robin T. Miller

Robin.Miller@gmail.com

Updated: September 23rd, 2025

Agenda

- Short History
- What is SPT?
- Why use this SPT Tool?
- SPT Features
- SPT Roadmap
- Q&A

Short History (mine, leading up to SPT)

- Started working with pSCSI in 1986 @Compugraphic
- Wrote SCSI subsystem (HBA, disk, tape drivers) for System V
- Developed streaming SCSI Tape driver for Solaris
- Enhanced Solaris *format* utility with extended SCSI commands
- Member of Tru64 SCSI/[CAM](#) (Common Access Method) team
 - Developed IOCTL interfaces, configuration code, tape library driver
- Developed SCSI CAM Utility (SCU) for Tru64 Unix @DEC
 - Ported SCU to multiple OS's for DEC Data Products Group
- Started developing SCSI pass-through (SPT) in 2006 @NetApp

SCSI Pass-Through (SPT) Utility

- SPT allows SCSI testing via the SCSI pass-through API
 - This is an OS specific user level API for issuing SCSI operations.
- Supported on most OS's, except OSX (lacks SCSI API)
- A fair amount of code originated from [DT tool](#)
- SPT is open source, and is available on GitHub:
 - <https://github.com/RobinTMiller/spt>
 - Note: Github does NOT include vendor extensions.
- Storage Enclosure Services (SES) support added

Why Use This SPT Tool?

- Advanced Testing Requires Advanced Test Tools
- Key SPT Features: (one tool versus many)
 - Specifically designed for SCSI testing
 - Supports multiple jobs/threads for high load & stress
 - Supports JavaScript Object Notation (JSON)
- Useful for developers, automation, and support.
- This TOI will hopefully whet your appetite!
- We have internal developer (Robin) and support! 😊

SPT Features

- Send/Receive Diagnostic Support (added for SES pages)
- Support for I/O Ranges (for read/write/verify operations)
- Support for I/O Modes (copy/verify/mirror operations)
- Support for Multiple Slices (threads for higher load & stress)
- Triage/Recovery Support (detailed errors, recovery, etc)
- Extended SCSI CDB Support (encode/decode CDB's)
- Enhanced Script File Support (custom test scripts)
- Emit Status Control Strings (roll your own output format)
- Test Check / Expect Options (automatic data/sense checks)
- Provides ASCII and JSON output formats! (work-in-progress)
- Support to unpack SCSI data (decode data in user format)

SPT Command Format

- Basic Command Format:

spt dsf=device cdb='hh hh ...' dir={none|read|write} length=value pout="hh hh ..." threads=value ...

- dsf is the device special file, e.g. /dev/sdXX or /dev/sgXX or /dev/dmXX (SCSI device)
- cdb = the Command Descriptor Block hex bytes (use 'spt showopcodes' to see operation codes)
- dir = the data direction (defaults to 'none')
- length = the data length
- pout = the parameter out data (optional)
- threads = the number of threads (optional, defaults to 1)

- Short Hand Commands: (for ease of use)

spt dsf=/dev/sg42 showhelp <- SES PP Help page

versus

spt dsf=/dev/sg42 cdb="1c 01 03 ff ff 00" dir=read length=65535 sname="Receive Diagnostic"

- I/O Parameters also Simplify CDB Generation:

spt dsf=/dev/sdl cdb=0x88 bs=64k starting=0 limit=1g slices=10 (READ (16) of 64k to limit of 1g of data)

Send/Receive Diagnostic Support

- Options Added for Storage Enclosure Services

Storage Enclosure Services (SES) Specific Options:

<code>element_index=value</code>	The element index.	(or <code>element=</code>)
<code>element_tcode=value</code>	The element type code.	(or <code>etcode=</code>)
<code>element_scode=value</code>	The element status code.	(or <code>escode=</code>)
<code>element_type=string</code>	The element type.	(or <code>etype=</code>)
<code>element_status=string</code>	The element status.	(or <code>estatus=</code>)
<code>rcvdiag</code>	Issue a receive diagnostic command.	
<code>senddiag</code>	Issue a send diagnostic command.	
<code>showhelp</code>	Show enclosure help text diagnostic page.	

Examples:

```
# spt rcvdiag page=configuration
```

Diagnostic Page Codes/Names

- Added Specifically for Storage Enclosure Devices

spt logprefix= rcvdiag page= (pages in **green** support decode functions)

Diagnostic Page Codes/Names:

- 0x00** - **Supported (supported)**
- 0x01** - **Configuration (configuration)**
- 0x02** - **Enclosure Control/Status (enclosure)**
- 0x03** - **Help (help)**
- 0x04** - **String In/Out (string)**
- 0x05 - Threshold In/Out (threshold)
- 0x07** - **Element Descriptor (element)**
- 0x08 - Short Enclosure Status (short_enclosure)
- 0x09 - Enclosure Busy (enclosure_busy)
- 0x0a** - **Additional Element Status (addl_enc_status)**
- 0x0b - Subenclosure Help Text (subenc_help_text)
- 0x0c - Subenclosure String In/Out (subenc_string)
- 0x0d - SES Diagnostic Pages (ses_diagnostic)
- 0x0e** - **Download Microcode Control/Status (download)**
- 0x0f - Subenclosure Nickname Control/Status (subenc_nickname)

Storage Enclosure Services (SES) Pages

- SES Pages Supported Today:
 - 0x00 = supported pages
 - 0x01 = configuration
 - 0x02 = enclosure status
 - 0x03 = help text (**showhelp**)
 - 0x04 = string in status
 - 0x07 = element descriptors
 - 0x0a = additional element status
 - 0x0e = download microcode
- Two output formats supported:
 - ASCII format (default)
 - JSON format (added for automation)

Supported Diagnostic Pages

```
# spt rcvdiag page=supported
```

```
Diagnostic Pages Supported:
```

Supported Page	(Code = 0x00)
Configuration Page	(Code = 0x01)
Enclosure Control/Status Page	(Code = 0x02)
Help Page	(Code = 0x03)
String In/Out Page	(Code = 0x04)
Threshold In/Out Page	(Code = 0x05)
Element Descriptor Page	(Code = 0x07)
Additional Element Status Page	(Code = 0x0a)
Download Microcode Control/Status Page	(Code = 0x0e)
Tag Data In/Out Page	(Code = 0x10)
Mini SAS HD Cable VPD Page	(Code = 0x17)

```
#
```

SES Configuration Page (PP Enclosure)

```
# spt rcvdiag page=config logprefix=
```

Configuration Diagnostic Page:

```

                                Page Code: 0x1
    Number of Secondary Enclosures: 0
                                Page Length: 244 (0xf4)
                                Generation Number: 0

    Enclosure Descriptor List:
    Number Enclosure Services Processes: 1
    Relative Enclosure Services Process ID: 1
        Subenclosure Identifier: 0 (Primary)
    Number of Type Descriptor Headers: 10
        Enclosure Descriptor Length: 36 (0x24)
        Enclosure Logical Identifier: 0x5000ccab00ff0108
        Vendor Identification: HGST
        Product Identification: STOR ENCL JBOD
        Product Revision Level: 0116

        Element Type: Array Device Slot (0x17)
        Number of Elements: 98
        Subenclosure Identifier: 0 (Primary)
        Text Length: 16 (0x10)
        Element Text: 'Array Slots'
        ...
```

SES Configuration Page (JSON format)

```
# spt rcvdiag page=config output-format=json logprefix=
{
  "Configuration": {
    "Length": 8,
    "Offset": 0,
    "Bytes": "01 00 00 f4 00 00 00 00",
    "Page Code": "0x01",
    "Secondary Enclosures": 0,
    "Page Length": 244,
    "Generation Number": 0,
    "Enclosure Descriptor List": [
      {
        "Length": 40,
        "Offset": 8,
        "Bytes": "11 00 0a 24 50 00 cc ab 00 ff 01 08 48 47 53 54 20 20
20 20 53 54 4f 52 20 45 4e 43 4c 20 4a 42 4f 44 20 20 30 31 31 36",
        "Enclosure Services Processes": 1,
        "Reserved (byte 0, bit 3)": 0,
        "Reserved (byte 0, bit 7)": 0,
        "Subenclosure Identifier": 0,
        "Type Descriptor Headers": 10,
        "Enclosure Descriptor Length": 36,
        "Enclosure Logical Identifier": "0x5000ccab00ff0108",
        "Vendor Identification": "HGST",
        "Product Identification": "STOR ENCL JBOD",
        "Product Revision Level": "0116"
      }
    ],
    ...
  }
}
```

SES Element Descriptor Page with Debug

```
# spt rcvdiag page=element enable=Debug logprefix=
```

```
Element Descriptor Diagnostic Page:
```

```

      Length: 8 (0x8)
Offset 0 (0x0): 07 00 16 32 00 00 00 00
      Page Code: 0x7
Reserved (byte 1): 0
      Page Length: 5682 (0x1632)
Generation Number: 0
...
Element Location: 230
      Length: 4 (0x4)
Offset 4966 (0x1366): 00 00 00 38
Element Descriptor: 0
      Element Text: IOM
Reserved (byte 0): 0
Reserved (byte 1): 0
Descriptor Length: 56 (0x38)
      Index: 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
Offset 4970 (0x136a): 45 53 43 45 20 49 4f 4d 20 20 20 20 20 20 20 20
      E S C E I O M
      Index: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
Offset 4986 (0x137a): 31 45 42 30 30 35 30 2d 33 41 20 20 20 20 20 20
      1 E B 0 0 5 0 - 3 A
      Index: 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
Offset 5002 (0x138a): 54 48 43 4c 53 30 34 35 31 35 52 49 30 30 31 42
      T H C L S 0 4 5 1 5 R I 0 0 1 B
      Index: 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
Offset 5018 (0x139a): 30 31 31 36 2d 30 30 31
      0 1 1 6 - 0 0 1
Descriptor Text: 'ESCE IOM 1EB0050-3A THCLS04515RI001B0116-001'
```

Built-In Support for SCSI Commands

- Added Built-in Options for Several SCSI Commands:
 - `inquiry {page=value}` Show Inquiry or specific page.
 - `logsense {page=value}` Show Log pages supported or page.
 - `zerolog {page=value}` Zero all Log pages or specific page.
 - `readcapacity16` Show disk capacity (16 byte CDB).
- Support based on automation requirements (so far).
- Support includes both ASCII and JSON output formats.
- FYI: When page value is omitted, displays page help.

Inquiry and Log Sense Pages

- Additional SCSI information added for Inquiry/Log Sense
 - Log Sense Protocol Specific Port page 0x18
 - Pages implemented are available in both ASCII and JSON
 - JSON avoids “screen scrapping”, and parses easily into Python dictionaries
 - JSON includes offset/hex bytes and all reserved/obsolete fields
 - This latter information can be used by automation for full verification
 - Note: Others to be added, when required and requested. 😊

Standard HGST Disk Inquiry (ASCII)

```
# spt dsf=/dev/sdcu inquiry logprefix=
```

```
Inquiry Information:
```

```
    Peripheral Device Type: 0 (Direct Access Device)
    Peripheral Qualifier: 0 (Peripheral Device Connected)
    Logical Unit Conglomerate: False
    Removable Media: False
    ANSI Version: 6 (Complies to ANSI INCITS 513 Revision 37a)
    Response Data Format: 2
    Historical Support: True
    Normal ACA Support: False
    Additional Length: 159 (0x9f)
    Supports Protection Information: True
    Third Party Copy Support: False
    Target Port Group Support: 0 (ALUA not supported)
    Storage Controller Components: False
    Multiple SCSI Ports: True
    Port Receiving Inquiry: 0 (Port A)
    Embedded Enclosure Services: False
    Command Queuing Support: True
    Vendor Identification: HGST
    Product Identification: HUH728080AL4200
    Firmware Revision Level: A7D8.
```

```
#
```

Standard HGST Disk Inquiry (JSON)

[illegible]

Acquiring HGST Full Firmware Version

SG Inquiry Method

```
# sg_inq --page=0x03 /dev/sd1
VPD INQUIRY: ASCII information page, FRU code=0x3
Vendor specific information in hex:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
10 00 00 00 4c 48 47 54 43 33 32 32 00 00 00 00 32 ...LHGTC322....2
20 30 30 31 43 33 32 32 30 30 30 01 33 32 32 00 31 001C322000.322.1
30 30 2f 31 39 2f 31 36 20 31 34 3a 30 35 3a 32 38 0/19/16 14:05:28
40 00 00 00 00 00 00 00 00 00 00 00 01 01 0e 24 4c .....$L
50 48 41 34 4b 00 00 00 45 4e 43 5f 30 36 47 5f 52 HA4K...ENC_06G_R
60 45 4c 45 41 53 45 00 62 6c 64 63 72 65 77 00 00 ELEASE.bldcrew..
70 00 00 00 66 6f 72 67 65 30 32 00 00 00 00 00 00 ...forge02.....
80 00 00 00 43 3a 2f 54 65 6d 70 2f 63 63 62 37 2f ...C:/Temp/ccb7/
90 52 33 32 32 2f 6c 69 62 72 61 72 79 2f 4c 48 41 R322/library/LHA
a0 34 4b 00 00 00 00 05 00 01 00 00 00 00 00 00 00 4K.....
b0 00 00 00 00 00 00 02 43 33 32 32 .....C322
#
```

Python Snippet: (old way)

```
message = "Find full firmware version for SAS drive"
command = "sudo {0}/sg_inq -p0x03 -H {1} | grep 10 | head -1 | awk -F '.'"
'{{print $NF}}}'".format(self._tools_directory, drive)
sg_inq_output = self._run_command(command=command, message=message,
logger=logger, shell=True)
firmware = sg_inq_output.strip()
```

Versus SPT JSON Method

```
# spt dsf=/dev/sd1 inquiry page=0x03 ofmt=json
{
  "ASCII Information": {
    "Length": 192,
    "Page Code": "0x03",
    "Page Length": 188,
    "Bytes": "00 03 00 bc 00 00 00 ..."
    ...
    "ASCII uCode Identifier": "LHGTC322",
    "ASCII Servo Part Number": "2001",
    ...
    "Code Mode Description": "OM_DISK - Drive is running code loaded from disk",
    "Flash Code Major Version": "C3",
    "Flash Code Minor Version": "22",
    "JSON Status": 0
  }
}
```

Python Snippet: (new way)

```
message = "Find full firmware version for SAS drive"
command = "{0}/spt dsf={1} inquiry page=ascii_info_page03 output-format=json"
emit="".format(self._tools_directory, disk_dev)
spt_json = self._run_command(command=command, message=message, logger=logger,
shell=False)
page03 = json.loads(spt_json)
current_firmware = page03['ASCII Information']['ASCII uCode Identifier'].strip()
```

Showing SCSI Log Sense Page Code/Names

```
# spt logprefix= logsense page= (pages in green support decode functions)
```

```
Log Page Codes/Names:
```

```
0x00 - Supported (supported)
0x01 - Buffer Overrun/Underrun Counter (overrun_underrun)
0x02 - Write Error Counter (write_error)
0x03 - Read Error Counter (read_error)
0x04 - Read Reverse Error Counter (read_reverse)
0x05 - Verify Error Counter (verify_error)
0x06 - Non-medium error counter (non-medium)
0x07 - Last n error events (last_error_events)
0x08 - Format Status (format_status)
0x0b - Last n Deferred or Async Events (last_deferred_events)
0x0a - Sequential-Access Device (sequential_access)
0x0c - Logical Block Provisioning (logical_block_provisioning)
0x0d - Temperature (temperature)
0x0e - Start-stop Cycle Counter (start_stop_cycle)
0x0f - Application Client (application_client)
0x10 - Self-Test Results (self_test_results)
0x11 - Solid State Media (solid_state_media)
0x15 - Background-Scan Results (background_scan)
0x17 - Non-Volatile Cache (non_volatile_cache)
0x18 - Protocol Specific Port (protocol_specific_port)
0x19 - Statistics and Performance (statistics_performance)
0x1a - SMART Status & Temp Reading (smart_status)
0x2f - Informational Exceptions (informational)
0x30 - Performance Counters (performance)
0x37 - Miscellaneous Data Counters (misc_data)
```

```
#
```

Log Sense Protocol Specific Page

```
# spt dsf=/dev/sddf logsense page=protocol_specific_port logprefix=
```

```
Protocol Specific Port Parameters (Page 0x18 - Current Cumulative Values):
```

```
        Page Code: 0x18
        Subpage Code: 0
        Page Length: 216 (0xd8)

        Parameter Code: 0x1 (Primary Port)
        Format and Linking: 3 (Binary format list)
        Target Save Disable (TSD): 0
        Parameter Length: 104 (0x68)

        Protocol Identifier: 0x6 (SAS Serial SCSI Protocol)
        Generation Code: 1
        Number of PHYs: 1

        SAS Phy Log Descriptor: 0
        Phy Identifier: 0
        SAS Phy Log Descriptor Length: 96 (0x60)
        Attached Reason: 8 (Phy Test Function Stopped)
        Attached Device Type: 2 (Expander device)
        Negotiated Physical Link Rate: 0xa (6 Gbps)
        Last Link Reset Reason: 0 (Power On)
        SMP Initiator Port: True
        STP Initiator Port: False
        SSP Initiator Port: False
        SMP Target Port: True
        STP Target Port: False
        SSP Target Port: False
        The SAS Address: 0x5000cca2510349b5
        Attached SAS Address: 0x5000ccab00ff010b
        Attached Phy Identifier: 13
        Invalid Dword Count: 0
        Running Disparity Error Count: 0
        Loss of Dword Synchronization: 0
        Phy Reset Problem: 0
        Phy Event Descriptor Length: 12 (0xc)
        Number of Event Descriptors: 4
        ...
```

SCSI Read Capacity (16) Example

```
# spt dsf=/dev/sddf logprefix= readcapacity16
```

```
Read Capacity(16) Data: (data length 32 bytes)
```

```
                Maximum Capacity: 2441609216 (9537536.000 megabytes)
                Block Length: 4096
                Protection Enabled: No
                Protection Type: 0
    Logical Blocks per Physical Exponent: 0 (1 blocks per physical)
                Protection Information Exponent: 0
    Logical Block Provisioning Management: No (Full Provisioned)
    Logical Block Provisioning Read Zeroes: No
    Lowest Aligned Logical Block Address: 0
```

```
# spt dsf=/dev/sddf logprefix= readcapacity16 output-format=json
```

```
{
  "Read Capacity(16)": {
    "Length": 32,
    "Offset": 0,
    "Bytes": "00 00 00 00 91 87 ff ff 00 00 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00",
    "Maximum Capacity": 2441609216,
    "Block Length": 4096,
    "Protection Enabled": false,
    "Protection Type": 0,
    "Logical Blocks per Physical Exponent": 0,
    "Protection Information Exponent": 0,
    "Logical Block Provisioning Management": false,
    "Logical Block Provisioning Read Zeroes": false,
    "Lowest Aligned Logical Block Address": 0,
    "JSON Status": 0
  }
}
#
```

Showing SCSI Operation Codes

\$ spt showopcodes						
Opcode	Subcode	Direction	Encode	Decode	Default	Opcode Name
0x40	0x00	none	no	no	0	Change Definition
0x39	0x00	none	no	no	0	Compare
0x18	0x00	none	no	no	0	Copy
0x3a	0x00	none	no	no	0	Copy and Verify
0x46	0x00	none	no	no	0	Get Configuration
0x12	0xff	read	yes	yes	0	Inquiry
0x12	0x00	none	no	no	0	Inquiry - Supported Pages
0x12	0x80	none	no	no	0	Inquiry - Serial Number
0x12	0x83	none	no	no	0	Inquiry - Device Identification
0x12	0x85	none	no	no	0	Inquiry - Management Network Addresses
0x12	0x86	none	no	no	0	Inquiry - Extended Inquiry Data
0x12	0x87	none	no	no	0	Inquiry - Mode Page Policy
0x12	0x8f	none	no	no	0	Inquiry - Third-party Copy
0x12	0xb0	none	no	no	0	Inquiry - Block Limits
0x12	0xb2	none	no	no	0	Inquiry - Logical Block Provisioning
0x4c	0x00	write	yes	no	0	Log Select
0x4d	0x00	read	yes	yes	0	Log Sense
0x15	0x00	none	no	no	0	Mode Select(6)
0x55	0x00	none	no	no	0	Mode Select(10)
0x1a	0x00	none	no	no	0	Mode Sense(6)
0x5a	0x00	none	no	no	0	Mode Sense(10)
0x3c	0x00	none	no	no	0	Read Buffer
0x1c	0x00	read	yes	yes	0	Receive Diagnostic
0x03	0x00	none	no	no	0	Request Sense
0x1d	0x00	write	yes	yes	0	Send Diagnostic
0x00	0x00	none	no	no	0	Test Unit Ready
...						
0x8a	0x00	write	yes	no	0	Write(16)
0x8e	0x00	write	yes	no	0	Write and Verify(16)
0x8f	0x00	none	yes	no	65536	Verify(16)
0x91	0x00	none	no	no	0	Synchronize Cache(16)
0x93	0x00	write	yes	no	65536	Write Same(16)
0x9e	0x00	read	no	no	0	Service Action In(16)
0x9e	0x10	read	yes	yes	0	Read Capacity(16)
0x9e	0x12	none	no	no	0	Get LBA Status(16)
0x89	0x00	none	no	no	0	Compare and Write(16)

Concurrent Testing of Multiple Devices

- Multiple Jobs via Script File (same as dt, but no job control)

```
# cat dospt-tur.sh
#!/bin/bash

disks=${disks:-""}
if [[ -z $disks ]];
then
    disks=$(lsscsi | fgrep disk | awk '{print $NF}')
    if [[ -z ${disks} ]]; then
        echo "Warning: No disks were found!"
        exit 1
    fi
fi

rm -f spt-tur.spt
for disk in ${disks}
do
    echo "dsf=${disk} cdb=0 status=good retry=100 msleep=100 enable=wait enable=async" >> spt-tur.spt
done
echo "sleep=3s" >> spt-tur.spt
spt script=spt-tur
exit $?
# ./dospt-tur.sh
# echo $?
0
#
```

Test Check Options

- Automatically Verify SCSI Status Information

Test Check Options:

resid=value	The expected residual.
transfer=value	The expected transfer count.
status=value	The expected SCSI status.
skey=value	The expected SCSI sense key.
asc=value	The expected SCSI sense code.
asq=value	The expected SCSI sense qualifier.

Note: The enable=wait option can be used to wait for status.

SCSI Status	Keyword	Value
-----	-----	-----
GOOD	good	0x00
CHECK_CONDITION	cc	0x02
CONDITION_MET	cmet	0x04
BUSY	busy	0x08
INTERMEDIATE	inter	0x10
INTER_COND_MET	icmet	0x14
RESERVATION_CONFLICT	rescon	0x18
COMMAND_TERMINATED	term	0x22
QUEUE_FULL	qfull	0x28
ACA_ACTIVE	aca_active	0x30
TASK_ABORTED	aborted	0x40

Example:

```
# spt cdb=0 status=good retry=100 msleep=100 enable=wait
```

Test Check Example

```
# lsscsi --generic | fgrep sdl
[0:0:11:0]    disk      HGST          HUH721010AL4204  C322  /dev/sdl      /dev/sg11
# spt dsf=/dev/sg11 op=lun_reset ; spt dsf=/dev/sg11 cdb=0 status=cc skey=6 asc=0x29 asq=0x3
# spt dsf=/dev/sg11 op=lun_reset ; spt dsf=/dev/sg11 cdb=0 status=good logprefix=
Result for Test Unit Ready
Expected:
    SCSI Status: 00h = SCSI_GOOD
    Sense Key: 00h = NO SENSE
    asc/ascq: 00h/00h = No additional sense information
Actual:
    SCSI Status: 02h = SCSI_CHECK_CONDITION
    Sense Key: 06h = UNIT ATTENTION
    asc/ascq: 29h/03h = Bus device reset function occurred
ERROR: Error number 1 occurred on Tue Oct 10 10:24:11 2017
ERROR: Unexpected response for SCSI opcode 0x0 (Test Unit Ready)

# echo $?
255
#
```

Expect Options

- Options to Automatically Verify Data Received

Expect Data Options:

`exp_radix={any,dec,hex}` The default is any radix.

`exp[ect]=type:byte_index:{string|value},...`

Where type is:

<code>C[HAR]</code>	Character strings to expect.
<code>B[YTE]</code>	Byte (8 bit) values to expect.
<code>S[SHORT]</code>	Short (16 bit) values to expect.
<code>W[ORD]</code>	Word (32 bit) values to expect.
<code>L[ONG]</code>	Long (64 bit) values to expect.

Note: Byte index and values are taken as decimal (by default).

Example Inquiry Verify:

```
# spt dsf=/dev/sg42 \
  cdb='12 00 00 00 ff 00' dir=read length=255 \
  expect=BYTE:0:0x0d,0x00,0x06,0x02,0x5b,0x00,0x40,0x02 \
  expect=C:8:'HGST      ','STOR ENCL JBOD  ' \
  expect=CHAR:32:'0116' disable=verbose
```

Please see Test Check Options for more test controls.

Expect Error Reporting

- Changed “HGST” to “WDC” to generate this error:

```
# spt dsf=/dev/sg42 \
    inquiry expect=BYTE:0:0x0d,0x00,0x06,0x02,0x5b,0x00,0x40,0x02 \
    expect=C:8:'WDC' , 'STOR ENCL JBOD' expect=CHAR:32:'0116' \
    disable=decode,verbose
spt (j:1 t:1): ERROR: Error number 1 occurred on Wed Oct 11 21:18:14 2017
spt (j:1 t:1): Failure while verifying Inquiry data on device /dev/sg42 (thread 1)
    -> Offset 000008: Expected Data: 87 (0x57, 'W') Received Data: 72 (0x48, 'H')
    -> Offset 000009: Expected Data: 68 (0x44, 'D') Received Data: 71 (0x47, 'G')
    -> Offset 000010: Expected Data: 67 (0x43, 'C') Received Data: 83 (0x53, 'S')
    -> Offset 000011: Expected Data: 32 (0x20, ' ') Received Data: 84 (0x54, 'T')
spt (j:1 t:1): Data Bytes: 96, Expect Entries: 36, Correct Entries: 32, Incorrect Entries: 4
# echo $?
255
#
```

Example SPT Script for SES Page Verification

```
#
# spt Script to verify MP SES Configuration Page 1.
#
$SPT_DEVICE=${SPT_DEVICE:? "Please define SPT_DEVICE for the /dev/sgNN device!"}
enable=scriptverify
exp_radix=hex

$VENDOR=${VENDOR:-'WDC          '}
$PRODUCT=${PRODUCT:-'InfiniFlash P200'}
$REVISION=${REVISION:-'0001'}
...
#
# SES Configuration Page 1:
#
rcvdiag page=configuration disable=decode,verbose transfer=248 \
#
# 00000 (0x0000)  01 00 00 f4 00 00 00 00
#                               Page Code: 0x1
#           Number of Secondary Enclosures: 0
#                               Page Length: 244
#                               Generation Number: 0
#
expect=BYTE:0:01,00,00,f4,00,00,00,00 \
#
# 00008 (0x0008)  11 00 0a 24 50 00 cc ab 05 01 01 80 57 44 43 20
# 00024 (0x0018)  20 20 20 20 49 6e 66 69 6e 69 46 6c 61 73 68 20
# 00040 (0x0028)  50 32 30 30 30 30 30 31
#           Enclosure Descriptor List:
#           Number Enclosure Services Processes: 1
#           Relative Enclosure Services Process ID: 1
#           Subenclosure Identifier: 0 (Primary)
#           Number of Type Descriptor Headers: 10
#           Enclosure Descriptor Length: 36
#           Enclosure Logical Identifier: 0x5000ccab05010180
#           Vendor Identification: WDC
#           Product Identification: InfiniFlash P200
#           Product Revision Level: 0001
#
expect=BYTE:8:11,00,0a,24,50,00,cc,ab,05,01,01,80 \
expect=CHAR:14:"${VENDOR}","${PRODUCT}","${REVISION}" \
...
```

Unpack SCSI Data Options

- A Format String to Unpack Various Data Received:

Unpack Data Options:

unpack=string The unpack format string.
unpack_fmt={dec,hex} The unpack data format. (Default: dec)

Where unpack format string is:

C[HAR]:[index]:length	Character of length.
F[IELD]:[index]:[start]:length	Extract bit field.
O[FFSET]:index	Set the buffer offset.
B[YTE][:index]	Decode byte (8 bit) value.
S[HORT][:index]	Decode short (16 bit) value.
W[ORD][:index]	Decode word (32 bit) values.
L[ONG][:index]	Decode long (64 bit) values.

Inquiry Unpack Examples:

```
# spt dsf=/dev/sdb inquiry disable=decode \  
unpack='Device Type: %BYTE, Vendor: %CHAR:8:8, Product: %C::16, Revision: %C::4\n'
```

OR Create your own JSON: (multiple unpack's permitted)

```
# spt dsf=/dev/sdb inquiry disable=decode \  
unpack='{ "Device Type": %BYTE, "Vendor": "%C:8:8",' \  
unpack=' "Product": "%C::16", "Revision": "%C::4" }\n'
```

Emit Status Control Strings

- SPT does NOT display statistics automatically.
- Roll-Your-Own Status/Statistics via: (or use **emit=default**)

```
# export SPT_EMIT_STATUS="
    Thread: %thread
    Device Name: %dsf
    SCSI Name: %scsi_name
    SCSI CDB: %cdb
    Iterations: %iterations
    Exit Status: %status = %status_msg
    Host Status: %host_status = %host_msg
    Driver Status: %driver_status = %driver_msg
    SCSI Status: %scsi_status = %scsi_msg
    Sense Code: %sense_code = %sense_msg
    Sense Key: %sense_key = %skey_msg
    asc/asq: (%asc, %asq) = %ascq_msg
    Transferred: %xfer (data bytes transferred)
    Residual: %resid (bytes not transferred)
    Total Blocks: %total_blocks
    Total Xferred: %total_xfer bytes
    Sense Data: %sense_data\n"
```

- See “spt help” for full list of Emit Status Control strings.

Example Emit Status Output

```
# spt dsf=/dev/sd1 cdb=0x8e starting=0 bs=64k limit=1g emit=default (Note: Writes all zero's, by default)
spt (j:1 t:1):
spt (j:1 t:1):          Thread: 1
spt (j:1 t:1):          Device Name: /dev/sd1
spt (j:1 t:1):          Device Info: Block Length=4096, Capacity=2441609216
spt (j:1 t:1):          Block Range: 0 - 262144
spt (j:1 t:1):          SCSI Name: Write and Verify(16)
spt (j:1 t:1):          SCSI CDB: 8e 00 00 00 00 00 00 03 ff 80 00 00 00 80 00 00
spt (j:1 t:1):          Data Direction: write
spt (j:1 t:1):          Data Length: 524288
spt (j:1 t:1):          Exit Status: 0 = SUCCESS
spt (j:1 t:1):          Host Status: 0 = DID_OK
spt (j:1 t:1):          Driver Status: 0 = DRIVER_OK
spt (j:1 t:1):          SCSI Status: 0 = SCSI_GOOD
spt (j:1 t:1):          Sense Code: 0 = NO_CODE
spt (j:1 t:1):          Sense Key: 0 = NO_SENSE
spt (j:1 t:1):          asc/asq: (00, 00) = No additional sense information
spt (j:1 t:1):          Bytes Transferred: 524288 (data bytes transferred)
spt (j:1 t:1):          Residual: 0 (bytes not transferred)
spt (j:1 t:1):          Iterations: 1
spt (j:1 t:1):          Total Bytes: 1073741824
spt (j:1 t:1):          Total Blocks: 262144
spt (j:1 t:1):          Total Operations: 2049
spt (j:1 t:1):          Sense Data: 00 00 00 00 00 00 00 00
spt (j:1 t:1):          Elapsed Time: 00m38.53s
spt (j:1 t:1):          Starting Time: Thu Oct 12 12:32:00 2017
spt (j:1 t:1):          Ending Time: Thu Oct 12 12:32:39 2017
spt (j:1 t:1):
#
```

Triage/Recovery Support

- Display of Full Sense Data via **enable=sense**
 - for xcopy, both copies of sense data displayed
- Automatic Recovery via **enable=recovery**
 - Error Conditions Retried:
 - Unit Attention, SCSI Busy, and Queue Full
 - Not Ready, in the process of becoming ready
 - Linux specific MPIO workaround
 - a read request is required to force path failover!
- Error Recovery Options:
 - recovery_delay=value - The amount of time to delay before retrying. (**Default: 2 secs**)
 - recovery_retries=value - The number of times to retry a SCSI request. (**Default: 60**)

Support for I/O Ranges

- I/O Ranges Automatically:
 - Sets the I/O direction
 - Adds enable=encode,decode (as required)
 - Automatically fills in the CDB fields
 - Optionally compares data read
- Support for I/O ranges for these CDB's
 - Read/Write (6, 10, and 16 byte)
 - Verify and Write Same (10 and 16 byte)
 - Unmap (frees blocks)

- Examples:

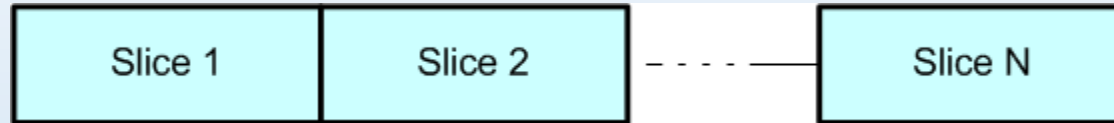
```
# export SPT_DEVICE=/dev/sdl
# SCSI_OPCODE_READ_16=0x88
# SCSI_OPCODE_WRITE_16=0x8a
# SCSI_OPCODE_WRITE_SAME_16=0x93
# spt cdb=${SCSI_OPCODE_WRITE_16} bs=32k starting=0 ptype=iot enable=sense
# spt cdb=${SCSI_OPCODE_READ_16} bs=32k starting=0 ptype=iot enable=compare
# spt cdb=${SCSI_OPCODE_WRITE_SAME_16} starting=0 blocks=16k enable=sense
```

Support for I/O Modes

- Similar to DT's I/O Modes
- I/O Mode Support:
 - iomode=mode - Set I/O mode to: {copy, mirror, **test**, or verify}
 - copy/verify added for copying and verifying LUNs
 - mirror mode added to verify synchronous mirrors
 - writing to one LUN should be mirrored to another LUN
- Examples:

```
# spt dsf=${SRC_DEVICE} dsf1=${DST_DEVICE} length=32k iomode=copy enable=compare
# spt dsf=${SRC_DEVICE} dsf1=${DST_DEVICE} length=32k iomode=verify
# spt dsf=${DST_DEVICE} dsf1=${DST_DEVICE} length=32k \
    writetype=writev16 iomode=mirror ptype=iot slices=10
```

Support for Multiple Slices



- Slices Create Multiple I/O Ranges
- Each Slice is a Thread with its' Own Range:

```
# SCSI_VERIFY_16=0x8f
# spt cdb=${SCSI_VERIFY_16} starting=0 slices=3 limit=1g \
  enable=Debug,sense,recovery logprefix=
Device: /dev/sdl, Device Size: 4096 bytes, Capacity: 2441609216 blocks (thread 0)
Device: /dev/sdl, Starting lba=0, ending lba=262143, block limit=262144 (thread 0)
Device: /dev/sdl, Starting lba=0, ending lba=87380, block limit=87381 (thread 1)
Device: /dev/sdl, Starting lba=87381, ending lba=174761, block limit=87381 (thread 2)
Device: /dev/sdl, Starting lba=174762, ending lba=262143, block limit=87382 (thread 3)
#
```

- Create lots of Slices to Generate Higher Load!

Support for Disruptive Testing

- Several popular methods include:
 - Setting incorrect CDB fields.
 - Sending unsupported CDB opcodes.
 - Sending incorrect I/O direction or length.
 - Sending high I/O to test Queue Full conditions.
 - Setting short timeouts to verify abort sequences.
 - On Linux, must use /dev/sgXX for short timeouts
 - Requests via /dev/sdXX (block device) enforce a minimum timeout:
 - linux/blkdev.h: #define **BLK_MIN_SG_TIMEOUT** (7 * HZ) (**7 seconds**)
- Sending Task Management Functions (TMF):
 - Bus Reset (may be simulated based on protocol)
 - LUN Reset
 - Target Reset

Example of Forcing Command Timeouts

```
# lsscsi -g | fgrep sdl
[0:0:11:0]    disk      HGST      HUH721010AL4204  C322  /dev/sdl      /dev/sg11
# spt dsf=/dev/sg11 cdb=${SCSI_VERIFY_16} starting=0 blocks=0xffff limit=1g timeout=1 logprefix=
ERROR: Error number 1 occurred on Wed Oct 11 16:21:14 2017
Verify(16) failed on device /dev/sg11 (thread 1)
SCSI CDB: 8f 00 00 00 00 00 00 00 00 00 00 00 ff ff 00 00, dir=none, length=0
SCSI Status = 0 (SCSI_GOOD)
Host Status = 0x3 (DID_TIME_OUT), Driver Status = 0 (DRIVER_OK)
Sense Key = 0 = NO SENSE, Sense Code/Qualifier = (0, 0) - No additional sense information
# dmesg --ctime | tail
...
[Wed Oct 11 16:17:32 2017] sd 0:0:11:0: attempting task abort! scmd(ffff881030c3b380)
[Wed Oct 11 16:17:32 2017] sd 0:0:11:0: [sg11] tag#0 CDB: Verify(16) 8f 00 00 00 00 00 00 00 00 00 00 00 ff ff 00 00
[Wed Oct 11 16:17:32 2017] scsi target0:0:11: handle(0x0015), sas_address(0x5000cca2510285c9), phy(11)
[Wed Oct 11 16:17:32 2017] scsi target0:0:11: enclosure_logical_id(0x5001636001caa0bd), slot(11)
[Wed Oct 11 16:17:32 2017] scsi target0:0:11: enclosure_level(0x0000),connector name(    )
[Wed Oct 11 16:17:32 2017] sd 0:0:11:0: task abort: SUCCESS scmd(ffff881030c3b380)
#
```

Extended SCSI CDB Support

- SCSI CDB's Extended by encode/decode
- I/O Options Enable this Automatically
- Read Capacity(16) Example:

```
# spt cdb='9e 10' dir=read length=32 enable=encode,decode disable=verbose
```

```
Read Capacity(16) Data: (data length 32 bytes)
```

```
Maximum Capacity: 204800 (100.000 megabytes)
```

```
Block Length: 512
```

```
Protection Enabled: No
```

```
Protection Type: 0
```

```
Logical Blocks per Physical Exponent: 3
```

```
Protection Information Exponent: 0
```

```
Thin Provisioning Enabled (LBPME): Yes
```

```
Thin Provisioning Read Zeroes (LBPRZ): Yes
```

```
Lowest Aligned Logical Block Address: 0
```

```
#
```

Extended SCSI CDB Support (continued...)

- SCSI Inquiry Low Level Equivalence:
 - inquiry = 'cdb='12 00 00 00 ff 00' dir=read length=255 enable=encode,decode disable=verbose

```
# spt inquiry logprefix=
```

```
Inquiry Information:
```

```
Peripheral Device Type: 0xd (Enclosure Services Device)
Peripheral Qualifier: 0 (Peripheral Device Connected)
Logical Unit Conglomerate: False
Removable Media: False
ANSI Version: 6 (Complies to ANSI INCITS 513 Revision 37a)
Response Data Format: 2
Historical Support: False
Normal ACA Support: False
Additional Length: 91 (0x5b)
Supports Protection Information: False
Third Party Copy Support: False
Target Port Group Support: 0 (ALUA not supported)
Storage Controller Components: False
Multiple SCSI Ports: False
Embedded Enclosure Services: True
Command Queuing Support: True
Vendor Identification: HGST
Product Identification: STOR ENCL JBOD
Firmware Revision Level: EL14
Detailed Firmware Revision: -004 04.08.01
Version Descriptor SAM-5: 0xa0
Version Descriptor SAS-2: 0xc20
Version Descriptor SPC-4: 0x460
Version Descriptor SES-3: 0x580
```

```
#
```

Enhanced Script File Support

- SPT Supports Script Files (5 levels deep)
- Environment Variables Now Expanded or Set:

```
# Enabling Debug shows us each spt command being executed.
enable=Debug
$SCSI_OPCODE_READ_16=0x88
$SCSI_OPCODE_WRITE_16=0x8a
$SCSI_OPCODE_WRITE_AND_VERIFY_16=0x8e
$SCSI_OPCODE_WRITE_SAME_16=0x93

$SLICES=3
$IO_LENGTH=64k
$ENABLE_OPTIONS="enable=Debug,sense"
$READ_OPTIONS="cdb=${SCSI_OPCODE_READ_16} dir=read length=${IO_LENGTH} ${ENABLE_OPTIONS}"
$WRITE_OPTIONS="cdb=${SCSI_OPCODE_WRITE_16} dir=write length=${IO_LENGTH} ${ENABLE_OPTIONS}"
$WRITE_SAME_OPTIONS="cdb=${SCSI_OPCODE_WRITE_SAME_16} starting=0 blocks=0xffff ${ENABLE_OPTIONS}"

# Initialize the source LUN with the IOT pattern.
dsf=${SRC_DEVICE} ${WRITE_OPTIONS} starting=0 ptype=iot slices=${SLICES}
# Zero the destination LUN using Write Same.
dsf=${DST_DEVICE} ${WRITE_SAME_OPTIONS}

# Copy from one LUN to another with data verification.
dsf=${SRC_DEVICE} dsf1=${DST_DEVICE} length=${IO_LENGTH} iomode=copy enable=compare,image emit=multi slices=10
...
```

SPT Roadmap

- Add more Inquiry, Log, and Mode pages.
- Add firmware download and Format support.
- Add JSON output format for other SCSI operations.
- Add SCSI compliance verification.
- Add support for Linux “bsg” SCSI interface.
- Add support for multiple devices. (like dt)
- Job Control for waiting on Async Threads. (like dt)
- Add server/client message passing using ZeroMQ.

Note: Not likely to be done now that I’m retired! ☹️